

Reviewer Pack — Minimal Rank of Tropicalized Configuration Spaces vs Randomi...

Entry 6057afa07a4f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 06:06:26 UTC

Minimal Rank of Tropicalized Configuration Spaces vs Randomized Communication Complexity for Disjointness

Entry ID: 6057afa07a4f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 06:06:26 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Configuration Space Theory) **Field B** (complexity object): Communication Complexity: Disjointness

Statement:

[‘For every instance of the disjointness problem with n variables, there exists a tropicalized configuration space invariant $\tau(\text{CS})$ associated with the instance such that $\tau(\text{CS})(\text{DISJ}_n) = \Omega(n^2)$, where DISJ_n is the disjointness function on n variables.’, ‘This lower bound holds for all possible distributions of inputs and random coin flips in the randomized communication protocol for disjointness.’]

Rationale (proposer’s reasoning):

[‘Tropicalized configuration spaces provide a geometric interpretation of combinatorial problems, allowing for the application of tools from algebraic geometry to study computational complexity.’, ‘The conjecture suggests that the structure of tropicalized configuration spaces could reveal hidden complexities in communication protocols, potentially leading to new lower bounds for disjointness.’]

Taxonomy category: COMM_DISJ (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 66f7116e8a8ba180

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all $n \leq 40$ and 30 random seeds, the minimal rank of the tropicalized configuration space $\tau(\text{CS})(\text{DISJ}_n)$ is at least $10n^2$ with 95% confidence interval.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            factor = Fraction(-A[i][i], A[i][i])
            for j in range(n):
                A[i][j] *= factor
            for j in range(m):
                if i != j:
                    factor = Fraction(A[j][i], A[i][i])
                    for k in range(n):
                        A[j][k] += factor * A[i][k]
        return A

    def matrix_multiplication(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[Fraction(0) for _ in range(p)] for _ in range(m)]
        for i in range(m):
            for j in range(n):
                for k in range(p):
                    C[i][k] += A[i][j] * B[j][k]
        return C
```

```

def rank(A):
    A = gaussian_elimination(A)
    r = 0
    for row in A:
        if any(row):
            r += 1
    return r

def generate_disjointness_instance(n):
    x = [random.randint(0, 1) for _ in range(n)]
    y = [random.randint(0, 1) for _ in range(n)]
    return x, y

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    total_rank = 0
    for _ in range(5): # Each n tested 5 times
        x, y = generate_disjointness_instance(n)
        A = [[x[i] + y[j]] for j in range(n)] for i in range(n)]
        total_rank += rank(A)

    avg_rank = Fraction(total_rank) / 5
    results.append({"n": n, "avg_rank": avg_rank})

metric_value = sum(result["avg_rank"] * result["n"]**2 for result in results) / sum(result["n"]**2 for result in results)
instances_tested = len(results)

conjecture_holds = all(result["avg_rank"] >= 10 * result["n"]**2 for result in results)
counterexample = "" if conjecture_holds else f"Failed for n={results[0]['n']}, avg_rank={results[0]['avg_rank']}"

return {
    "metric_name": "Minimal Rank of Tropicalized Configuration Space",
    "metric_value": float(metric_value),
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):

```

```

    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Failed for n={results[0]['n']}\", avg_rank={result['avg_rank']}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_36fa74d4.py", line 94, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_36fa74d4.py", line 69, in run_trial
    total_rank += rank(A)
                  ^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_36fa74d4.py", line 49, in rank
    A = gaussian_elimination(A)
        ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_36fa74d4.py", line 29, in gaussian_elimination
    factor = Fraction(-A[i][i], A[i][i])
              ^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
ZeroDivisionError: Fraction(0, 0)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not complete the required number of trials to verify the conjecture's support condition. | next: Re-run the test with increased robustness checks and ensure that it completes without crashing. If the crash is resolved, re-evaluate the results against the pre-registered support conditions.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14569
2	propose	ollama_remote	glm4:latest	0	0	13320
3	preregistration	ollama_remote	glm4:latest	0	0	9369
4	novelty	ollama_remote	glm4:latest	0	0	8750
5	novelty	ollama_remote	glm4:latest	0	0	8657
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11064
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7511
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9169
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12094
10	judge	ollama_remote	glm4:latest	0	0	11852

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 106356 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/6057afa07a4f.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/6057afa07a4f.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/6057afa07a4f.tar.gz* (if generated)